

```
!pip install opencv-python scikit-learn matplotlib numpy
```

Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/
 Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/d
 Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dis
 Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-pac
 Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/d
 Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/
 Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/pyth
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.
 Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.13/d
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.1
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/pyth
 Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

print("Libraries loaded successfully!")
```

Libraries loaded successfully!

```
from google.colab import files

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

# OpenCV BGR ko RGB mein convert karna
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print("Image loaded successfully!")
print("Image size:", img.shape)
```

iron man.jpeg

iron man.jpeg(image/jpeg) - 880580 bytes, last modified: 9/15/2026 - 100% done
 Saving iron man.jpeg to iron man.jpeg
 Image loaded successfully!
 Image size: (1448, 898, 3)

```
plt.figure(figsize=(8, 10))
plt.imshow(img_rgb)
plt.title("Original Image")
```

```
plt.axis("off")  
plt.show()
```

Original Image



```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
  
plt.figure(figsize=(8, 10))  
plt.imshow(gray, cmap="gray")  
plt.title("Grayscale Image")  
plt.axis("off")  
plt.show()
```

Grayscale Image



```
# Gaussian Blur
blur = cv2.GaussianBlur(gray, (15, 15), 0)

plt.figure(figsize=(14, 7))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Grayscale Image



Gaussian Blurred Image



```
ret, global_thresh = cv2.threshold(
    blur,
    127,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(8, 10))
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()

print("Threshold value used:", ret)
```

Global Thresholding



Threshold value used: 127.0

```
otsu_value, otsu_thresh = cv2.threshold(  
    blur,  
    0,  
    255,  
    cv2.THRESH_BINARY + cv2.THRESH_OTSU  
)  
  
plt.figure(figsize=(8, 10))  
plt.imshow(otsu_thresh, cmap="gray")
```

```
plt.title("Otsu's Thresholding")  
plt.axis("off")  
plt.show()  
  
print("Automatically selected Otsu threshold:", otsu_value)
```

Otsu's Thresholding



Automatically selected Otsu threshold: 114.0

```
adaptive_thresh = cv2.adaptiveThreshold(  
    blur,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
```

```
cv2.THRESH_BINARY,  
11,  
2  
)  
  
plt.figure(figsize=(8, 10))  
plt.imshow(adaptive_thresh, cmap="gray")  
plt.title("Adaptive Thresholding")  
plt.axis("off")  
plt.show()
```

Adaptive Thresholding



```
# Binary image
_, thresh = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)
```

```
# Noise removal
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)
```

```
# Background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)
```

```
# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)
```

```
# Foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)
```

```
# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)
```

```
# Marker labelling
num_labels, markers = cv2.connectedComponents(sure_fg)
```

```
markers = markers + 1

markers[unknown == 255] = 0
```

```
# Apply Watershed
watershed_bgr = img.copy()

markers = cv2.watershed(
    watershed_bgr,
    markers
)

# Convert result to RGB
watershed_rgb = cv2.cvtColor(
    watershed_bgr,
    cv2.COLOR_BGR2RGB
)

# Mark watershed boundaries in RED
watershed_rgb[markers == -1] = [255, 0, 0]
```

```
plt.figure(figsize=(8, 10))
plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```

Watershed Segmentation



```
# Resize image for faster processing
scale_width = 500

scale_height = int(
    img_rgb.shape[0] * scale_width / img_rgb.shape[1]
)

small_img = cv2.resize(
    img_rgb,
    (scale_width, scale_height)
```

```
)
```

```
# Convert pixels into data
pixel_data = small_img.reshape((-1, 3))

pixel_data = np.float32(pixel_data)

# Number of clusters
K = 3
```

```
# K-Means
kmeans = KMeans(
    n_clusters=K,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixel_data)

# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Replace pixels with cluster colors
segmented_data = centers[labels]
```

```
# Reshape back to image
kmeans_image = segmented_data.reshape(
    small_img.shape
)
```

```
print("\n=====")
print(" IMAGE SEGMENTATION RESULTS")
print("=====")

print("Original Image Size:", img.shape)
print("Otsu Automatically Selected Threshold:", otsu_value)
print("K-Means Number of Clusters:", K)

# Create figure
plt.figure(figsize=(18, 20))

# 1 Original
plt.subplot(4, 2, 1)
plt.imshow(img_rgb)
plt.title("1. Original Image", fontsize=14)
plt.axis("off")

# 2 Grayscale
plt.subplot(4, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("2. Grayscale Image", fontsize=14)
plt.axis("off")
```

```
# 3 Gaussian Blur
plt.subplot(4, 2, 3)
plt.imshow(blur, cmap="gray")
plt.title("3. Gaussian Blurred Image", fontsize=14)
plt.axis("off")

# 4 Global Threshold
plt.subplot(4, 2, 4)
plt.imshow(global_thresh, cmap="gray")
plt.title("4. Global Thresholding", fontsize=14)
plt.axis("off")

# 5 Otsu
plt.subplot(4, 2, 5)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("5. Otsu's Thresholding", fontsize=14)
plt.axis("off")

# 6 Adaptive
plt.subplot(4, 2, 6)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("6. Adaptive Thresholding", fontsize=14)
plt.axis("off")

# 7 Watershed
plt.subplot(4, 2, 7)
plt.imshow(watershed_rgb)
plt.title("7. Watershed Segmentation", fontsize=14)
plt.axis("off")

# 8 K-Means
plt.subplot(4, 2, 8)
plt.imshow(kmeans_image)
plt.title("8. K-Means Segmentation (K=3)", fontsize=14)
plt.axis("off")

plt.tight_layout()

# Save combined result
plt.savefig(
    "Experiment_6_All_Results.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```
=====
IMAGE SEGMENTATION RESULTS
=====
Original Image Size: (1448, 898, 3)
Otsu Automatically Selected Threshold: 114.0
K-Means Number of Clusters: 3
```

1. Original Image



2. Grayscale Image



```
def save_rgb_image(image, filename, title):
    plt.figure(figsize=(7, 10))
    plt.imshow(image)
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()

    plt.savefig(
        filename,
        dpi=300,
        bbox_inches="tight",
        pad_inches=0.1
    )

    plt.close()

def save_gray_image(image, filename, title):
    plt.figure(figsize=(7, 10))
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()

    plt.savefig(
        filename,
        dpi=300,
        bbox_inches="tight",
        pad_inches=0.1
    )

    plt.close()
```

```
save_rgb_image(
    img_rgb,
    "01_Original.png",
    "Original Image"
)

save_gray_image(
    gray,
    "02_Grayscale.png",
    "Grayscale Image"
)
```

```
save_gray_image(
    blur,
    "03_Gaussian_Blur.png",
    "Gaussian Blurred Image"
)

save_gray_image(
    global_thresh,
    "04_Global_Threshold.png",
    "Global Thresholding"
)

save_gray_image(
    otsu_thresh,
    "05_Otsu_Threshold.png",
    "Otsu's Thresholding"
)

save_gray_image(
    adaptive_thresh,
    "06_Adaptive_Threshold.png",
    "Adaptive Thresholding"
)

save_rgb_image(
    watershed_rgb,
    "07_Watershed.png",
    "Watershed Segmentation"
)

save_rgb_image(
    kmeans_image,
    "08_KMeans.png",
    "K-Means Segmentation"
)
```

```
import zipfile
import os

files_to_zip = [
    "Experiment_6_All_Results.png",
    "01_Original.png",
    "02_Grayscale.png",
    "03_Gaussian_Blur.png",
    "04_Global_Threshold.png",
    "05_Otsu_Threshold.png",
    "06_Adaptive_Threshold.png",
    "07_Watershed.png",
    "08_KMeans.png"
]

with zipfile.ZipFile(
    "Experiment_6_Results.zip",
    "w"
```

```
) as zipf:

    for file in files_to_zip:
        zipf.write(file)

print("\n=====")
print(" EXPERIMENT COMPLETED!")
print("=====")

print("\nGenerated files:")

for file in files_to_zip:
    print("✓", file)

print("\nOtsu Threshold =", otsu_value)
print("K-Means Clusters =", K)

print("\nAll images have been saved at 300 DPI "\n

=====
```